

Agentic-OS

Dario Anchundia

Las entidades representadas en el proyecto son: las ventanas gráficas (**x11_client**), el proceso *Launcher* y el servidor *IAlearner*. Debido a que la estación de trabajo del usuario y el servidor de clasificación son entidades físicamente separadas, se optó por sockets TCP para el intercambio de datos entre ambos, reservando las señales POSIX (**SIGCHLD**) únicamente para el control local entre el Launcher y sus procesos hijos. Cada tecla presionada en una ventana se envía carácter por carácter al servidor, que las agrupa en palabras y oraciones, las clasifica mediante *Bag of Words* y, al finalizar todos los procesos, infiere el contexto de usuario.

Agentic-OS — Diagrama de despliegue

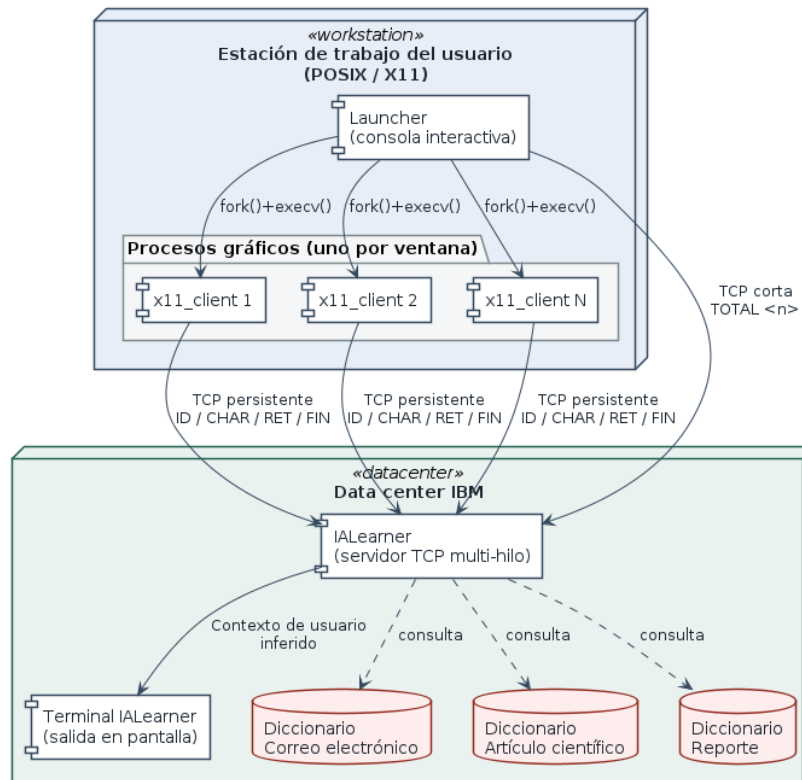


Figure 1: Diagrama de despliegue de Agentic-OS

Componentes

Launcher (proceso, consola interactiva):

1. Valida los parámetros de entrada (**N**, host, puerto).
2. Notifica al IAlearner, mediante una conexión TCP corta, el mensaje **TOTAL <n>** con el número de ventanas que debe esperar.
3. Crea cada proceso gráfico (**x11_client**) mediante `fork()` y `execv()`.
4. Despliega un menú interactivo para monitorear o cerrar los procesos, y define el manejador de **SIGCHLD** con `waitpid(-1,&estado,WNOHANG)` en un **while**, garantizando cero procesos zombie sin bloquear el menú.

x11_client (un proceso por ventana):

1. Se conecta por TCP al IAlearner y se identifica con **ID <n>**.
2. Envía cada tecla imprimible mediante **CHAR <c>** en ráfagas asíncronas, sin bloquear el hilo de dibujo gráfico.
3. Al presionar Enter envía **RET** (fin de oración); al presionar Escape envía **FIN** y cierra la ventana.

IAlearner (servidor TCP multi-hilo):

1. Acepta múltiples conexiones y atiende cada una en un hilo independiente (`pthread_create`).
2. Cada ventana tiene su propio vector de frecuencias protegido por su propio `pthread_mutex_t`, evitando condiciones de carrera al sumar frecuencias.
3. Un hilo clasificador permanece en `pthread_cond_wait` (sin espera activa) y solo despierta cuando el contador de ventanas terminadas coincide con el **TOTAL** esperado.
4. Toda impresión en consola pasa por un mutex global (`g_mutex_consola`) para evitar que se entrelacen caracteres de distintos hilos.

Protocolo interno (IPC vía sockets TCP)

Cada línea enviada por socket termina en salto de línea. Mensajes definidos en `protocolo.h`:

Mensaje	Dirección	Significado
ID <n>	ventana → IALearner	La ventana <i>n</i> se identifica al conectar
CHAR <c>	ventana → IALearner	Se presionó la tecla imprimible <i>c</i>
RET	ventana → IALearner	Fin de palabra/oración (Enter)
FIN	ventana → IALearner	La ventana se cerró (Escape)
TOTAL <n>	Launcher → IALearner	Cuántas ventanas esperar en la ronda

Clasificación (Bag of Words)

Un documento (ventana) se clasifica en una clase si aparecen al menos **3 coincidencias** de su diccionario. Si califica para más de una clase, se asigna la de mayor frecuencia total acumulada.

Correo electrónico	Artículo científico	Reporte
thank, please, regards, meeting, attached, information, update, schedule, team, project	data, analysis, results, method, study, model, research, system, significant, effect	system, data, network, security, application, server, user, performance, service, infrastructure

Una vez que terminan todos los procesos esperados, se evalúa qué clases resultaron ganadoras y se compara contra la tabla de perfiles (X = presencia binaria de al menos un documento con esa etiqueta):

Tipo de usuario	Correo	Artículo	Reporte
Personal administrativo	X		
Personal técnico	X		X
Profesor	X	X	
Estudiante		X	X

Concurrencia y programación defensiva

El servidor IALearner acepta conexiones concurrentes con un hilo por ventana, cada uno protegido por su propio mutex, lo que permite procesar varias ventanas en paralelo sin bloquearse entre sí. El hilo clasificador evita la espera activa mediante `pthread_cond_wait`, despertando únicamente cuando se cumple la condición de cierre de ronda. En el Launcher, el manejador de SIGCHLD recolecta de inmediato los recursos de los procesos hijos apenas el usuario cierra una ventana (Escape), garantizando cero procesos zombie sin congelar el menú interactivo. Toda escritura en la consola del servidor está serializada mediante un mutex global para evitar entrelazado de caracteres entre hilos.

Compilación y ejecución

```
gcc ia_learner/ia_learner.c -o ia_learner/ia_learner -Wall -Wextra -lpthread
gcc launcher/launcher.c -o launcher/launcher -Wall -Wextra
gcc x11_client/x11_client.c -o x11_client/x11_client -Wall -Wextra -lX11
```

Terminal 1 (servidor, se arranca siempre primero): `./ia_learner [puerto]` (por defecto 9500). Terminal 2 (launcher): `./launcher/launcher <N-ventanas> [host] [puerto]`. El launcher notifica TOTAL <n> al IALearner y abre las *N* ventanas gráficas; al terminar todas, el IALearner muestra en pantalla el contexto de usuario inferido.